

will be committed without any further action on your part. This is a very convenient feature, but it can also be dangerous. It's a good idea to always check your commits carefully to be sure you aren't committing any files that you didn't intend to modify. When TortoiseSVN displays its Commit Files dialog, you can double-click on an individual file in order to see the diffs you made prior to hitting the "OK" button.

2.1.8 Deleting Files

When a file is deleted from the repository, it's not really gone. The file still exists in the repository, but its latest version is simply marked "deleted" so that users will no longer see the file in their local directory trees. You can still see and access previous versions of a deleted file by right-clicking on the folder in which the file was contained and selecting "Show log" from the TortoiseSVN menu.

You can undelete a deleted file by updating your local directory to the version immediately before the version in which the file was marked deleted. Then simply commit the file again. This replaces the latest deleted version of the file with the version just prior to the deletion, effectively undeleting the file.

2.2 Compilers, Linkers and IDEs

Compiled languages, such as C++, require a *compiler* and *linker* in order to transform source code into an executable program. There are many compilers/linkers available for C++, but for the Microsoft Windows platform, the most commonly used package is probably Microsoft Visual Studio. The fully-featured Professional and Enterprise Editions of the product can be purchased online from the Microsoft store, and its lighter-weight cousin Visual Studio Community Edition (previously known as Visual Studio Express) is available for free download at <https://www.visualstudio.com/downloads>. Documentation on Visual Studio and the standard C and C++ libraries is available online at the Microsoft Developer Network (MSDN) site (<https://msdn.microsoft.com/en-us>).

Visual Studio is more than just a compiler and linker. It is an *integrated development environment* (IDE), including a slick and fully featured *text editor* for source code and a powerful source-level and machine-level *debugger*. In this book, our primary focus is the Windows platform, so we'll investigate Visual Studio in some depth. Much of what you learn below will be applicable to other compilers, linkers and debuggers such as LLVM/Clang, gcc/gdb, and

the Intel C/C++ compiler. So even if you're not planning on ever using Visual Studio, I suggest you read this section carefully. You'll find all sorts of useful tips on using compilers, linkers and debuggers in general.

2.2.1 Source Files, Headers and Translation Units

A program written in C++ is comprised of *source files*. These typically have a .c, .cc, .cxx or .cpp extension, and they contain the bulk of your program's source code. Source files are technically known as *translation units*, because the compiler *translates* one source file at a time from C++ into machine code.

A special kind of source file, known as a *header file*, is often used in order to share information, such as type declarations and function prototypes, between translation units. Header files are not seen by the compiler. Instead, the C++ *preprocessor* replaces each `#include` statement with the contents of the corresponding header file prior to sending the translation unit to the compiler. This is a subtle but very important distinction to make. Header files exist as distinct files from the point of view of the programmer—but thanks to the preprocessor's header file expansion, all the compiler ever sees are translation units.

2.2.2 Libraries, Executables and Dynamic Link Libraries

When a translation unit is compiled, the resulting machine code is placed in an object file (files with a .obj extension under Windows or .o under UNIX-based operating systems). The machine code in an object file is:

- *relocatable*, meaning that the memory addresses at which the code resides have not yet been determined, and
- *unlinked*, meaning that any external references to functions and global data that are defined outside the translation unit have not yet been resolved.

Object files can be collected into groups called *libraries*. A library is simply an archive, much like a ZIP or tar file, containing zero or more object files. Libraries exist merely as a convenience, permitting a large number of object files to be collected into a single easy-to-use file.

Object files and libraries are *linked* into an executable by the linker. The executable file contains fully resolved machine code that can be loaded and run by the operating system. The linker's jobs are:

- to calculate the final relative addresses of all the machine code, as it will appear in memory when the program is run, and
- to ensure that all external references to functions and global data made by each translation unit (object file) are properly resolved.

It's important to remember that the machine code in an executable file is still relocatable, meaning that the addresses of all instructions and data in the file are still *relative* to an arbitrary base address, not absolute. The final absolute base address of the program is not known until the program is actually loaded into memory, just prior to running it.

A *dynamic link library* (DLL) is a special kind of library that acts like a hybrid between a regular static library and an executable. The DLL acts like a library, because it contains functions that can be called by any number of different executables. However, a DLL also acts like an executable, because it can be loaded by the operating system independently, and it contains some start-up and shut-down code that runs much the way the `main()` function in a C++ executable does.

The executables that use a DLL contain *partially linked* machine code. Most of the function and data references are fully resolved within the final executable, but any references to external functions or data that exist in a DLL remain unlinked. When the executable is run, the operating system resolves the addresses of all unlinked functions by locating the appropriate DLLs, loading them into memory if they are not already loaded, and patching in the necessary memory addresses. Dynamically linked libraries are a very useful operating system feature, because individual DLLs can be updated without changing the executable(s) that use them.

2.2.3 Projects and Solutions

Now that we understand the difference between libraries, executables and dynamic link libraries (DLLs), let's see how to create them. In Visual Studio, a *project* is a collection of *source files* which, when compiled, produce a library, an executable or a DLL. In all versions of Visual Studio since VS 2013, projects are stored in project files with a `.vcxproj` extension. These files are in XML format, so they are reasonably easy for a human to read and even edit by hand if necessary.

All versions of Visual Studio since version 7 (Visual Studio 2003) employ *solution files* (files with a `.sln` extension) as a means of containing and managing collections of projects. A solution is a collection of dependent and/or independent projects intended to build one or more libraries, executables and/or DLLs. In the Visual Studio graphical user interface, the *Solution Explorer* is

Modern compilers provide so many options that it would be impractical and error prone to specify all of them every time you build your code. That's where *build configurations* come in. A build configuration is really just a collection of preprocessor, compiler and linker options associated with a particular project in your solution. You can define any number of build configurations, name them whatever you want, and configure the preprocessor, compiler and linker options differently in each configuration. By default, the same options are applied to every translation unit in the project, although you can override the global project settings on an individual translation unit basis. (I recommend avoiding this if at all possible, because it becomes difficult to tell which .cpp files have custom settings and which do not.)

When you create a new project/solution in Visual Studio, by default it creates two build configurations named "Debug" and "Release." The release build is intended for the final version of your software that you'll ship (release) to customers, while the debug build is for development purposes. A debug build runs more slowly than a non-debug build, but it provides the programmer with invaluable information for developing and debugging the program.

Professional software developers often set up more than two build configurations for their software. To understand why, we'll need to understand how local (compile-time) and global (link-time) optimizations work—we'll discuss these kinds of optimizations in Section 2.2.4.2. For now, let's drop the somewhat confusing term "release build" and stick with the terms "debug build" (meaning local and global optimizations are disabled) and "non-debug build" (meaning local and/or global optimizations are enabled).

2.2.4.1 Common Build Options

This section lists some of the most common options you'll want to control via build configurations for a game engine project.

Preprocessor Settings

The C++ preprocessor handles the expansion of `#included` files and the definition and substitution of `#defined` macros. One extremely powerful feature of all modern C++ preprocessors is the ability to define preprocessor macros via command-line options (and hence via build configurations). Macros defined in this way act as though they had been written into your source code with a `#define` statement. For most compilers, the command line option for this is `-D` or `/D`, and any number of these directives can be used.

This feature allows you to communicate various build options to your code,

without having to modify the source code itself. As a ubiquitous example, the symbol `_DEBUG` is always defined for a debug build, while in non-debug builds, the symbol `NDEBUG` is defined instead. The source code can check these flags and in effect “know” whether it is being built in debug or non-debug mode. This is known as *conditional compilation*. For example,

```
void f()
{
#ifdef _DEBUG
    printf("Calling function f()\n");
#endif
    // ...
}
```

The compiler is also free to introduce “magic” preprocessor macros into your code, based on its knowledge of the compilation environment and target platform. For example, the macro `__cplusplus` is defined by most C/C++ compilers when compiling a C++ file. This allows code to be written that automatically adapts to being compiled for C or C++.

As another example, every compiler identifies itself to the source code via a “magic” preprocessor macro. When compiling code under the Microsoft compiler, the macro `_MSC_VER` is defined; when compiling under the GNU compiler (gcc), the macro `__GNUC__` is defined instead and so on for the other compilers. The target platform on which the code will be run is likewise identified via macros. For example, when building for a 32-bit Windows machine, the symbol `_WIN32` is always defined. These key features permit cross-platform code to be written, because they allow your code to “know” what compiler is compiling it and on which target platform it is destined to be run.

Compiler Settings

One of the most common compiler options controls whether or not the compiler should include *debugging information* with the object files it produces. This information is used by debuggers to step through your code, display the values of variables and so on. Debugging information makes your executables larger on disk and also opens the door for hackers to reverse-engineer your code, so it is always stripped from the final shipping version of your executable. However, during development, debugging information is invaluable and should always be included in your builds.

The compiler can also be told whether or not to expand *inline functions*. When inline function expansion is turned off, every inline function appears only once in memory, at a distinct address. This makes the task of tracing

through the code in the debugger much simpler, but obviously comes at the expense of the execution speed improvements normally achieved by inlining.

Inline function expansion is but one example of generalized code transformations known as *optimizations*. The aggressiveness with which the compiler attempts to optimize your code, and the kinds of optimizations it uses, can be controlled via compiler options. Optimizations have a tendency to reorder the statements in your code, and they also cause variables to be stripped out of the code altogether, or moved around, and can cause CPU registers to be reused for new purposes later in the same function. Optimized code usually confuses most debuggers, causing them to “lie” to you in various ways, and making it difficult or impossible to see what’s really going on. As a result, all optimizations are usually turned off in a debug build. This permits every variable and every line of code to be scrutinized as it was originally coded. But, of course, such code will run much more slowly than its fully optimized counterpart.

Linker Settings

The linker also exposes a number of options. You can control what type of output file to produce—an executable or a DLL. You can also specify which external libraries should be linked into your executable, and which directory paths to search in order to find them. A common practice is to link with debug libraries when building a debug executable and with optimized libraries when building in non-debug mode.

Linker options also control things like stack size, the preferred base address of your program in memory, what type of machine the code will run on (for machine-specific optimizations), whether or not global (link-time) optimizations are enabled, and some other minutia with which we will not concern ourselves here.

2.2.4.2 Local and Global Optimizations

An optimizing compiler is one that can automatically optimize the machine code it generates. All of the C/C++ compilers commonly used today are optimizing compilers. These include Microsoft Visual Studio, gcc, LLVM/Clang and the Intel C/C++ compiler.

Optimizations can come in two basic forms:

- local optimizations and
- global optimizations,

although other kinds of optimizations are possible (e.g., *peep-hole optimizations*, which enable the optimizer to make platform- or CPU-specific optimizations).

Local optimizations operate only on small chunks of code known as *basic blocks*. Roughly speaking, a basic block is a sequence of assembly language instructions that doesn't involve a branch. Local optimizations include things like:

- *algebraic simplification*,
- *operator strength reduction* (e.g., converting $x / 2$ into $x \gg 1$ because the shift operator is “lower strength” and therefore less expensive than the integer division operator),
- *code inlining*,
- *constant folding* (recognizing expressions that are constant at compile time and replacing such expressions with their known value),
- *constant propagation* (replacing all instances of a variable whose value turns out to be constant with the literal constant itself),
- *loop unrolling* (e.g., converting a loop that always iterates exactly four times with four copies of the code within the loop, in order to eliminate conditional branching),
- *dead code elimination* (removal of code that has no effect, such as removing the assignment expression $x = 5$; if it is immediately followed by another assignment to x like $x = y + 1$), and
- *instruction reordering* (in order to minimize CPU pipeline stalls).

Global optimizations operate beyond the scope of basic code blocks—they take the whole control flow graph of a program into account. An example of this kind of optimization would be *common sub-expression elimination*. Global optimizations ideally operate across translation unit boundaries, and hence are performed by the linker rather than the compiler. Aptly, optimizations performed by the linker are known as *link-time optimizations* (LTO).

Some modern compilers like LLVM/Clang support *profile-guided optimizations* (PGO). As the name implies, these optimizations use profiling information obtained from previous runs of your software to iteratively identify and optimize its most performance-critical code paths. PGO and LTO optimizations can produce an impressive performance boost, but they come with a cost. LTO optimizations greatly increase the amount of time required to link an executable. And PGO optimizations, being iterative in nature, require the software to be run (via the QA team or an automated test suite) in order to generate the profiling information that drives further optimizations.

Most compilers offer various options controlling how aggressive its optimization efforts will be. Optimizations can be disabled entirely (useful for debug builds, where debuggability is more important than performance), or increasing “levels” of optimizations can be applied up to some predetermined maximum. For example, optimizations for gcc, Clang and Visual C++ all range from -O0 (meaning no optimizations are performed) to -O3 (all optimizations enabled). Individual optimizations can be turned on or off as well, via other command line options.

2.2.4.3 Typical Build Configurations

Game projects often have more than just two build configurations. Here are a few of the common configurations I’ve seen used in game development.

- *Debug.* A debug build is a very slow version of your program, with all optimizations turned off, all function inlining disabled, and full debugging information included. This build is used when testing brand new code and also to debug all but the most trivial problems that arise during development.
- *Development.* A development build (or “dev build”) is a faster version of your program, with most or all local optimizations enabled, but with debugging information and assertions still turned on. (See Section 3.2.3.3 for a discussion of assertions.) This allows you to see your game running at a speed representative of the final product, but it still gives you some opportunity to debug problems.
- *Ship.* A ship configuration is intended for building the final game that you will ship to your customers. It is sometimes called a “final” build or “disk” build. Unlike a development build, all debugging information is stripped out of a ship build, most or all assertions are compiled out, and optimizations are cranked all the way up, including global optimizations (LTO and PGO). A ship build is very tricky to debug, but it is the fastest and leanest of all build types.

Hybrid Builds

A hybrid build is a build configuration in which the majority of the translation units are built in development mode, but a small subset of them is built in debug mode. This permits the segment of code that is currently under scrutiny to be easily debugged, while the rest of the code continues to run at close to full speed.

With a text-based build system like make, it is quite easy to set up a hybrid build that permits users to specify the use of debug mode on a per-

translation-unit basis. In a nutshell, we define a make variable called something like `$HYBRID_SOURCES`, which lists the names of all translation units (.cpp files) that should be compiled in debug mode for our hybrid build. We set up build rules for compiling both debug and development versions of every translation unit, and arrange for the resulting object files (.obj/.o) to be placed into two different folders, one for debug and one for development. The final link rule is set up to link with the debug versions of the object files listed in `$HYBRID_SOURCES` and with the non-debug versions of all other object files. If we've set it up properly, make's dependency rules will take care of the rest.

Unfortunately, this is not so easy to do in Visual Studio, because its build configurations are designed to be applied on a per-project basis, not per-translation unit. The crux of the problem is that we cannot easily define a list of the translation units that we want to build in debug mode. One solution to this problem is to write a script (in Python or another suitable language) that automatically generates Visual Studio .vcxproj files, given a list of the .cpp files you want to be built in debug mode in your hybrid configuration. Another alternative that works if your source code is already organized into libraries is to set up a "Hybrid" build configuration at the solution level, which picks and chooses between debug and development builds on a per-project (and hence per-library) basis. This isn't as flexible as having control on a per-translation-unit basis, but it does work reasonably well if your libraries are sufficiently granular.

Build Configurations and Testability

The more build configurations your project supports, the more difficult testing becomes. Although the differences between the various configurations may be slight, there's a finite probability that a critical bug may exist in one of them but not in the others. Therefore, each build configuration must be tested equally thoroughly. Most game studios do not formally test their debug builds, because the debug configuration is primarily intended for internal use during initial development of a feature and for the debugging of problems found in one of the other configurations. However, if your testers spend most of their time testing your development build configuration, then you cannot simply make a ship build of your game the night before Gold Master and expect it to have an identical bug profile to that of the development build. Practically speaking, the test team must test both your development and ship builds equally throughout alpha and beta to ensure that there aren't any nasty surprises lurking in your ship build. In terms of testability, there is a clear advantage to keeping your build configurations to a minimum, and in

fact some studios don't have a separate ship build for this reason—they simply ship their *development* build once it has been thoroughly tested (but with the debugging information stripped out).

2.2.4.4 Project Configuration Tutorial

Right-clicking on any project in the Solution Explorer and selecting “Properties...” from the menu brings up the project's “Property Pages” dialog. The tree view on the left shows various categories of settings. Of these, the four we will use most are:

- Configuration Properties/General,
- Configuration Properties/Debugging,
- Configuration Properties/C++, and
- Configuration Properties/Linker.

Configurations Drop-Down Combo Box

Notice the drop-down combo box labeled “Configuration:” at the top-left corner of the window. All of the properties displayed on these property pages apply separately to each build configuration. If you set a property for the debug configuration, this does not necessarily mean that the same setting exists for the other configuration(s).

If you click on the combo box to drop down the list, you'll find that you can select a single configuration or multiple configurations, including “All configurations.” As a rule of thumb, try to do most of your build configuration editing with “All configurations” selected. That way, you won't have to make the same edits multiple times, once for each configuration—and you don't risk setting things up incorrectly in one of the configurations by accident. However, be aware that some settings *need* to be different between the debug and development configurations. For example, function inlining and code optimization settings should, of course, be different between debug and development builds.

General Property Page

On the *General* property page, shown in Figure 2.9, the most useful fields are the following:

- *Output directory*. This defines where the final product(s) of the build will go—namely, the executable, library or DLL that the compiler/linker ultimately outputs.

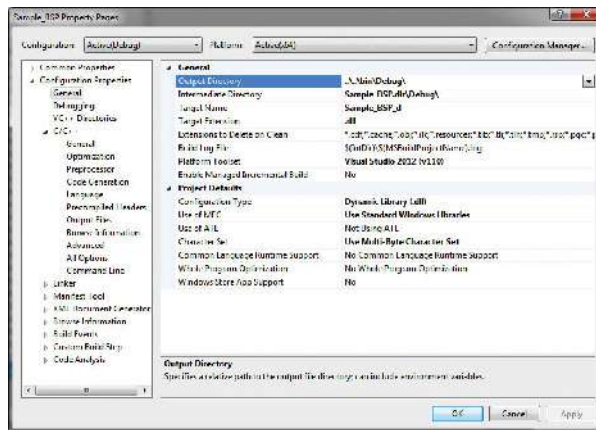


Figure 2.9. Visual Studio project property pages—General page.

- *Intermediate directory.* This defines where intermediate files, primarily object files (.obj extension), are placed during a build. Intermediate files are never shipped with your final program—they are only required during the process of building your executable, library or DLL. Hence, it is a good idea to place intermediate files in a different directory than the final products (.exe, .lib or .dll files).

Note that VisualStudio provides a macro facility, which may be used when specifying directories and other settings in the “Project Property Pages” dialog. A *macro* is essentially a named variable that contains a global value and that can be referred to in your project configuration settings.

Macros are invoked by writing the name of the macro enclosed in parentheses and prefixed with a dollar sign (e.g., `$ (ConfigurationName)`). Some commonly used macros are listed below.

- `$ (TargetFileName)`. The name of the final executable, library or DLL file being built by this project.
- `$ (TargetPath)`. The full path of the folder containing the final executable, library or DLL.
- `$ (ConfigurationName)`. The name of the build config, which will be “Debug” or “Release” by default in Visual Studio, although as we’ve said, a real game project will probably have multiple configurations such as “Debug,” “Hybrid,” “Development” and “Ship.”
- `$ (OutDir)`. The value of the “Output Directory” field specified in this dialog.

- `$(IntDir)`. The value of the “Intermediate Directory” field in this dialog.
- `$(VCInstallDir)`. The directory in which Visual Studio’s C++ standard library is currently installed.

The benefit of using macros instead of hard-wiring your configuration settings is that a simple change of the global macro’s value will automatically affect all configuration settings in which the macro is used. Also, some macros like `$(ConfigurationName)` automatically change their values depending on the build configuration, so using them can permit you to use identical settings across all your configurations.

To see a complete list of all available macros, click in either the “Output Directory” field or the “Intermediate Directory” field on the “General” property page, click the little arrow to the right of the text field, select “Edit...” and then click the “Macros” button in the dialog that comes up.

Debugging Property Page

The “Debugging” property page is where the name and location of the executable to debug is specified. On this page, you can also specify the command-line argument(s) that should be passed to the program when it runs. We’ll discuss debugging your program in more depth below.

C/C++ Property Page

The C/C++ property page controls compile-time language settings—things that affect how your source files will be compiled into object files (.obj extension). The settings on this page do *not* affect how your object files are linked into a final executable or DLL.

You are encouraged to explore the various subpages of the C/C++ page to see what kinds of settings are available. Some of the most commonly used settings include the following:

- *General Property Page/Additional Include Directories*. This field lists the on-disk directories that will be searched when looking for `#included` header files.

Important: It is always best to specify these directories using *relative paths* and/or with Visual Studio macros like `$(OutDir)` or `$(IntDir)`. That way, if you move your build tree to a different location on disk or to another computer with a different root folder, everything will continue to work properly.

- *General Property Page/Debug Information Format.* This field controls whether or not debug information is generated and in what format. Typically both debug and development configurations include debugging information so that you can track down problems during the development of your game. The ship build will have all the debug info stripped out to prevent hacking.
- *Preprocessor Property Page/Preprocessor Definitions.* This very handy field lists any number of C/C++ preprocessor symbols that should be defined in the code when it is compiled. See *Preprocessor Settings* in Section 2.2.4.1 for a discussion of preprocessor-defined symbols.

Linker Property Page

The “Linker” property page lists properties that affect how your object code files will be linked into an executable or DLL. Again, you are encouraged to explore the various subpages. Some commonly used settings follow:

- *General Property Page/Output File.* This setting lists the name and location of the final product of the build, usually an executable or DLL.
- *General Property Page/Additional Library Directories.* Much like the C/C++ Additional Include Directories field, this field lists zero or more directories that will be searched when looking for libraries and object files to link into the final executable.
- *Input Property Page/Additional Dependencies.* This field lists external libraries that you want linked into your executable or DLL. For example, the OGRE libraries would be listed here if you are building an OGRE-enabled application.

Note that Visual Studio employs various “magic spells” to specify libraries that should be linked into an executable. For example, a special `#pragma` instruction in your source code can be used to instruct the linker to automatically link with a particular library. For this reason, you may not see all of the libraries you’re actually linking to in the “Additional Dependencies” field. (In fact, that’s why they are called *additional* dependencies.) You may have noticed, for example, that Direct X applications do not list all of the DirectX libraries manually in their “Additional Dependencies” field. Now you know why.

2.2.5 Debugging Your Code

One of the most important skills any programmer can learn is how to effectively debug code. This section provides some useful debugging tips and

tricks. Some are applicable to any debugger and some are specific to Microsoft Visual Studio. However, you can usually find an equivalent to Visual Studio's debugging features in other debuggers, so this section should prove useful even if you don't use Visual Studio to debug your code.

2.2.5.1 The Start-Up Project

A Visual Studio solution can contain more than one project. Some of these projects build executables, while others build libraries or DLLs. It's possible to have more than one project that builds an executable in a single solution. Visual Studio provides a setting known as the "Start-Up Project." This is the project that is considered "current" for the purposes of the debugger. Typically a programmer will debug one project at a time by setting a single start-up project. However, it is possible to debug multiple projects simultaneously (see [http://msdn.microsoft.com/en-us/library/0s590bew\(v=vs.100\).aspx](http://msdn.microsoft.com/en-us/library/0s590bew(v=vs.100).aspx) for details).

The start-up project is highlighted in bold in the Solution Explorer. By default, hitting F5 will run the .exe built by the start-up project, if the start-up project builds an executable. (Technically speaking, F5 runs whatever command you type into the Command field in the Debugging property page, so it's not limited to running the .exe built by your project.)

2.2.5.2 Breakpoints

Breakpoints are the bread and butter of code debugging. A breakpoint instructs the program to stop at a particular line in your source code so that you can inspect what's going on.

In Visual Studio, select a line and hit F9 to toggle a breakpoint. When you run your program and the line of code containing the breakpoint is about to be executed, the debugger will stop the program. We say that the breakpoint has been "hit." A little arrow will show you which line of code the CPU's program counter is currently on. This is shown in Figure 2.10.

2.2.5.3 Stepping through Your Code

Once a breakpoint has been hit, you can single-step your code by hitting the F10 key. The yellow program-counter arrow moves to show you the lines as they execute. Hitting F11 steps *into* a function call (i.e., the next line of code you'll see is the first line of the called function), while F10 steps *over* that function call (i.e., the debugger calls the function at full speed and then breaks again on the line right after the call).

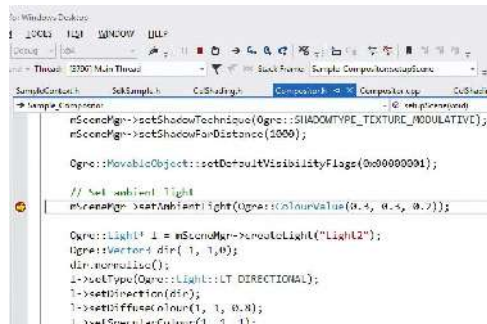


Figure 2.10. Setting a breakpoint in Visual Studio.

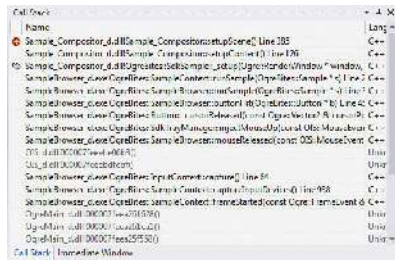


Figure 2.11. The call stack window.

2.2.5.4 The Call Stack

The *call stack* window, shown in Figure 2.11, shows you the stack of functions that have been called at any given moment during the execution of your code. (See Section 3.3.5.2 for more details on a program's stack.) To display the call stack (if it is not already visible), go to the “Debug” menu on the main menu bar, select “Windows” and then “Call Stack.”

Once a breakpoint has been hit (or the program is manually paused), you can move up and down the call stack by double-clicking on entries in the “Call Stack” window. This is very useful for inspecting the chain of function calls that were made between `main()` and the current line of code. For example, you might trace back to the root cause of a bug in a parent function that has manifested itself in a deeply nested child function.

2.2.5.5 The Watch Window

As you step through your code and move up and down the call stack, you will want to be able to inspect the values of the variables in your program. This is what *watch windows* are for. To open a watch window, go to the “Debug” menu, select “Windows...,” then select “Watch...,” and finally select one of

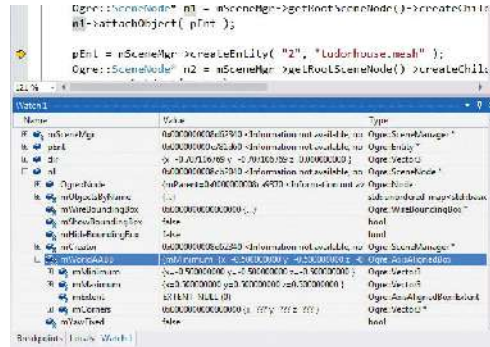


Figure 2.12. Visual Studio's watch window.

“Watch 1” through “Watch 4.” (Visual Studio allows you to open up to four watch windows simultaneously.) Once a watch window is open, you can type the names of variables into the window or drag expressions in directly from your source code.

As you can see in Figure 2.12, variables with simple data types are shown with their values listed immediately to the right of their names. Complex data types are shown as little tree views that can be easily expanded to “drill down” into virtually any nested structure. The base class of a class is always shown as the first child of an instance of a derived class. This allows you to inspect not only the class’ data members, but also the data members of its base class(es).

You can type virtually any valid C/C++ *expression* into the watch window, and Visual Studio will evaluate that expression and attempt to display the resulting value. For example, you could type “5 + 3” and Visual Studio will display “8.” You can cast variables from one type to another by using C or C++ casting syntax. For example, typing “(float) intVar1 / (float) intVar2” in the watch window will display the ratio of two integer variables as a floating-point value.

You can even *call functions in your program* from within the watch window. Visual Studio reevaluates the expressions typed into the watch window(s) automatically, so if you invoke a function in the watch window, it will be called every time you hit a breakpoint or single-step your code. This allows you to leverage the functionality of your program in order to save yourself work when trying to interpret the data that you’re inspecting in the debugger. For example, let’s say that your game engine provides a function called `quatToAngleDeg()`, which converts a quaternion to an angle of rotation in degrees. You can call this function in the watch window in order to easily

inspect the rotation angle of any quaternion within the debugger.

You can also use various suffixes on the expressions in the watch window in order to change the way Visual Studio displays the data, as shown in Figure 2.13.

- The “,d” suffix forces values to be displayed in decimal notation.
- The “,x” suffix forces values to be displayed in hexadecimal notation.
- The “,n” suffix (where *n* is any positive integer) forces Visual Studio to treat the value as an array with *n* elements. This allows you to expand array data that is referenced through a pointer.
- You can also write simple expressions in square brackets that *calculate* the value of *n* in a “,n” suffix. For example, you can type something like this:

```
my_array, [my_array_count]
```

to ask the debugger to show `my_array_count` elements of the array named `my_array`.

Be careful when expanding very large data structures in the watch window, because it can sometimes slow the debugger down to the point of being unusable.

2.2.5.6 Data Breakpoints

Regular breakpoints trip when the CPU’s program counter hits a particular machine instruction or line of code. However, another incredibly useful feature of modern debuggers is the ability to set a breakpoint that trips whenever a specific memory address is *written to* (i.e., changed). These are called *data breakpoints*, because they are triggered by changes to data, or sometimes *hardware breakpoints*, because they are implemented via a special feature of the CPU’s hardware—namely, the ability to raise an interrupt when a predefined memory address is written to.

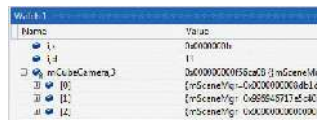


Figure 2.13. Comma suffixes in the Visual Studio watch window.

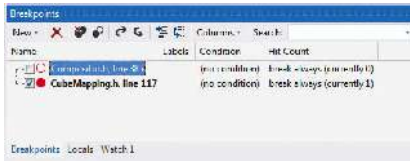


Figure 2.14. The Visual Studio breakpoints window.

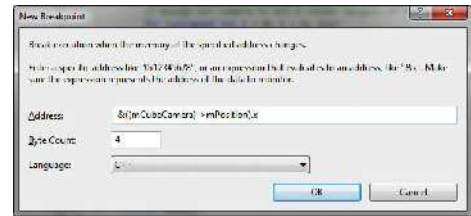


Figure 2.15. Defining a data breakpoint.

Here's how data breakpoints are typically used. Let's say you are tracking down a bug that manifests itself as a zero (0.0f) value mysteriously appearing inside a member variable of a particular object called `m_angle` that *should* always contain a nonzero angle. You have no idea which function might be writing that zero into your variable. However, you do know the address of the variable. (You can just type "`&object.m_angle`" into the watch window to find its address.) To track down the culprit, you can set a data breakpoint on the address of `object.m_angle`, and then simply let the program run. When the value changes, the debugger will stop automatically. You can then inspect the call stack to catch the offending function red-handed.

To set a data breakpoint in Visual Studio, take the following steps.

- Bring up the "Breakpoints" window found on the "Debug" menu under "Windows" and then "Breakpoints" (Figure 2.14).
- Select the "New" drop-down button in the upper-left corner of the window.
- Select "New Data Breakpoint."
- Type in the raw address or an address-valued expression, such as "`&myVariable`" (Figure 2.15).

2.2.5.7 Conditional Breakpoints

You'll also notice in the "Breakpoints" window that you can set conditions and hit counts on any type breakpoint—data breakpoints or regular line-of-code breakpoints.

A *conditional breakpoint* causes the debugger to evaluate the C/C++ expression you provide every time the breakpoint is hit. If the expression is true, the debugger stops your program and gives you a chance to see what's going on. If the expression is false, the breakpoint is ignored and the program continues. This is very useful for setting breakpoints that only trip when a function is called on a particular instance of a class. For example,

let's say you have a game level with 20 tanks on-screen, and you want to stop your program when the third tank, whose memory address you know to be 0x12345678, is running. By setting the breakpoint's condition expression to something like `"(uintptr_t)this == 0x12345678"`, you can restrict the breakpoint only to the class instance whose memory address (`this` pointer) is 0x12345678.

Specifying a *hit count* for a breakpoint causes the debugger to decrement a counter every time the breakpoint is hit, and only actually stop the program when that counter reaches zero. This is really useful for situations where your breakpoint is inside a loop, and you need to inspect what's happening during the 376th iteration of the loop (e.g., the 376th element in an array). You can't very well sit there and hit the F5 key 375 times! But you *can* let the hit count feature of Visual Studio do it for you.

One note of caution: conditional breakpoints cause the debugger to evaluate the conditional expression every time the breakpoint is hit, so they can bog down the performance of the debugger and your game.

2.2.5.8 Debugging Optimized Builds

I mentioned above that it can be very tricky to debug problems using a development or ship build, due primarily to the way the compiler optimizes the code. Ideally, every programmer would prefer to do all of his or her debugging in a debug build. However, this is often not possible. Sometimes a bug occurs so rarely that you'll jump at any chance to debug the problem, even if it occurs in a non-debug build on someone else's machine. Other bugs only occur in your non-debug builds, but they magically disappear whenever you run the debug build. These dreaded *non-debug-only bugs* are sometimes caused by uninitialized variables, because variables and dynamically allocated memory blocks are often set to zero in debug mode but are left containing garbage in a non-debug build. Other common causes of non-debug-only bugs include code that has been accidentally omitted from the non-debug build(s) (e.g., when important code is erroneously placed inside an assertion statement), data structures whose size or data member packing changes between debug and development/ship builds, bugs that are only triggered by inlining or compiler-introduced optimizations, and (in rare cases) bugs in the compiler's optimizer itself, causing it to emit incorrect code in a fully optimized build.

Clearly, it behooves every programmer to be capable of debugging problems in a non-debug build, unpleasant as it may seem. The best ways to reduce the pain of debugging optimized code is to practice doing it and to expand your skill set in this area whenever you have the opportunity. Here are a few tips.

- *Learn to read and step through disassembly in the debugger.* In a non-debug build, the debugger often has trouble keeping track of which line of source code is currently being executed. Thanks to instruction reordering, you'll often see the program counter jump around erratically within the function when viewed in source code mode. However, things become sane again when you work with the code in disassembly mode (i.e., step through the assembly language instructions individually). Every C/C++ programmer should be at least a little bit familiar with the architecture and assembly language of their target CPU(s). That way, even if the debugger is confused, you won't be. (For an introduction to assembly language, see Section 3.4.7.3.)
- *Use registers to deduce variables' values or addresses.* The debugger will sometimes be unable to display the value of a variable or the contents of an object in a non-debug build. However, if the program counter is not too far away from the initial use of the variable, there's a good chance its address or value is still stored in one of the CPU's registers. If you can trace back through the disassembly to where the variable is first loaded into a register, you can often discover its value or its address by inspecting that register. Use the register window, or type the name of the register into a watch window, to see its contents.
- *Inspect variables and object contents by address.* Given the address of a variable or data structure, you can usually see its contents by casting the address to the appropriate type in a watch window. For example, if we know that an instance of the `Foo` class resides at address `0x1378A0C0`, we can type "`(Foo*) 0x1378A0C0`" in a watch window, and the debugger will interpret that memory address as if it were a pointer to a `Foo` object.
- *Leverage static and global variables.* Even in an optimized build, the debugger can usually inspect global and static variables. If you cannot deduce the address of a variable or object, keep your eye open for a static or global that might contain its address, either directly or indirectly. For example, if we want to find the address of an internal object within the physics system, we might discover that it is in fact stored in a member variable of the global `PhysicsWorld` object.
- *Modify the code.* If you can reproduce a non-debug-only bug relatively easily, consider modifying the source code to help you debug the problem. Add print statements so you can see what's going on. Introduce a global variable to make it easier to inspect a problematic variable or

object in the debugger. Add code to detect a problem condition or to isolate a particular instance of a class.

2.3 Profiling Tools

Games are typically high-performance real-time programs. As such, game engine programmers are always looking for ways to speed up their code. In this section, we'll investigate some of the tools we can use to measure the performance of our software. We'll have more to say about how to use this profiling data in order to optimize our software in Chapter 4.

There is a well-known, albeit rather unscientific, rule of thumb known as the *Pareto principle* (see http://en.wikipedia.org/wiki/Pareto_principle). It is also known as the *80/20 rule*, because it states that in many situations, 80% of the effects of some event come from only 20% of the possible causes. In computer science, this principle has been applied both to bug-fixing (80% of the perceived bugs in a piece of software can be eliminated by fixing bugs in only 20% of the code), and to software optimization, where as a rule of thumb 80% (or more) of the wall clock time spent running any piece of software is accounted for by only 20% (or less) of the code. In other words, if you optimize 20% of your code, you can potentially realize 80% of all the gains in execution speed you'll ever realize.

So, how do you know *which* 20% of your code to optimize? For that, you need a *profiler*. A profiler is a tool that measures the execution time of your code. It can tell you how much time is spent in each function. You can then direct your optimizations toward only those functions that account for the lion's share of the execution time.

Some profilers also tell you how many *times* each function is called. This is an important dimension to understand. A function can eat up time for two reasons: (a) it takes a long time to execute on its own, or (b) it is called frequently. For example, a function that runs an A* algorithm to compute the optimal paths through the game world might only be called a few times each frame

Even more information can be obtained if you use the right profiler. Some profilers report the call graph, meaning that for any given function, you can see which functions called it (these are known as *parent functions*) and which functions it called (these are known as *child functions* or *descendants*). You can even see what *percentage* of the function's time was spent calling each of its descendants and the percentage of the overall running time accounted for by each individual function.

Profilers fall into two broad categories.

1. *Statistical profilers.* This kind of profiler is designed to be unobtrusive, meaning that the target code runs at almost the same speed, whether or not profiling is enabled. These profilers work by sampling the CPU's program counter register periodically and noting which function is currently running. The number of samples taken within each function yields an approximate percentage of the total running time that is eaten up by that function. Intel's *VTune* is the gold standard in statistical profilers for Windows machines employing Intel Pentium processors, and it is now also available for Linux. See <https://software.intel.com/en-us/intel-vtune-amplifier-xe> for details.
2. *Instrumenting profilers.* This kind of profiler is aimed at providing the most accurate and comprehensive timing data possible, but at the expense of real-time execution of the target program—when profiling is turned on, the target program usually slows to a crawl. These profilers work by preprocessing your executable and inserting special prologue and epilogue code into every function. The prologue and epilogue code calls into a profiling library, which in turn inspects the program's call stack and records all sorts of details, including which parent function called the function in question and how many times that parent has called the child. This kind of profiler can even be set up to monitor every line of code in your source program, allowing it to report how long each line is taking to execute. The results are stunningly accurate and comprehensive, but turning on profiling can make a game virtually unplayable. IBM's Rational Quantify, available as part of the Rational Purify Plus tool suite, is an excellent instrumenting profiler. See <http://www.ibm.com/developerworks/rational/library/957.html> for an introduction to profiling with Quantify.

Microsoft has also published a profiler that is a hybrid between the two approaches. It is called LOP, which stands for low-overhead profiler. LOP uses a statistical approach, sampling the state of the processor periodically, which means it has a low impact on the speed of the program's execution. However, with each sample, it analyzes the call stack, thereby determining the chain of parent functions that resulted in each sample. This allows LOP to provide information normally not available with a statistical profiler, such as the distribution of calls across parent functions.

On the PlayStation 4, SN Systems' *Razor CPU* is the profiler of choice for measuring game software running on the PS4's CPU. It supports both statis-

tical and instrumenting methods of profiling. (See <https://www.snsystems.com/tech-blog/2014/02/14/function-level-profiling/> for some more details.) Its counterpart, *Razor GPU*, provides profiling and debugging facilities for shaders and compute jobs running on the PS4's GPU.

2.3.1 List of Profilers

There are a great many profiling tools available. See http://en.wikipedia.org/wiki/List_of_performance_analysis_tool for a reasonably comprehensive list.

2.4 Memory Leak and Corruption Detection

Two other problems that plague C and C++ programmers are memory leaks and memory corruption. A memory leak occurs when memory is allocated but never freed. This wastes memory and eventually leads to a potentially fatal out-of-memory condition. Memory corruption occurs when the program inadvertently writes data to the wrong memory location, overwriting the important data that was there—while simultaneously failing to update the memory location where that data *should* have been written. Blame for both of these problems falls squarely on the language feature known as the *pointer*.

A pointer is a powerful tool. It can be an agent of good when used properly—but it can also be all-too-easily transformed into an agent of evil. If a pointer points to memory that has been freed, or if it is accidentally assigned a nonzero integer or floating-point value, it becomes a dangerous tool for corrupting memory, because data written through it can quite literally end up anywhere. Likewise, when pointers are used to keep track of allocated memory, it is all too easy to forget to free the memory when it is no longer needed. This leads to memory leaks.

Clearly, good coding practices are one approach to avoiding pointer-related memory problems. And it is certainly possible to write solid code that essentially never corrupts or leaks memory. Nonetheless, having a tool to help you detect potential memory corruption and leak problems certainly can't hurt. Thankfully, many such tools exist.

My personal favorite is IBM's Rational Purify, which comes as part of the Purify Plus toolkit. Purify instruments your code prior to running it, in order to hook into all pointer dereferences and all memory allocations and deallocations made by your code. When you run your code under Purify, you get a live report of the problems—real and potential—encountered by your code. And when the program exits, you get a detailed memory leak report. Each problem is linked directly to the source code that caused the problem, mak-

ing tracking down and fixing these kinds of problems relatively easy. You can find more information on Purify at <http://www-306.ibm.com/software/awdtools/purify>.

Two other popular tools are Insure++ by Parasoft, and Valgrind by Julian Seward and the Valgrind development team. These tools provide both memory debugging and profiling facilities.

2.5 Other Tools

There are a number of other commonly used tools in a game programmer's toolkit. We won't cover them in any depth here, but the following list will make you aware of their existence and point you in the right direction if you want to learn more.

- *Difference tools.* A difference tool, or *diff tool*, is a program that compares two versions of a text file and determines what has changed between them. (See <http://en.wikipedia.org/wiki/Diff> for a discussion of diff tools.) Diffs are usually calculated on a line-by-line basis, although modern diff tools can also show you a range of characters on a changed line that have been modified. Most version control systems come with a diff tool. Some programmers like a particular diff tool and configure their version control software to use the tool of their choice. Popular tools include ExamDiff (http://www.prestosoft.com/edp_examdiff.asp), AraxisMerge (<http://www.araxis.com>), WinDiff (available in the Options Packs for most Windows versions and available from many independent websites as well), and the GNU diff tools package (<http://www.gnu.org/software/diffutils/diffutils.html>).
- *Three-way merge tools.* When two people edit the same file, two independent sets of diffs are generated. A tool that can merge two sets of diffs into a final version of the file that contains both person's changes is called a three-way merge tool. The name "three-way" refers to the fact that three versions of the file are involved: the original, user A's version and user B's version. (See http://en.wikipedia.org/wiki/3-way_merge#Three-way_merge for a discussion of two-way and three-way merge technologies.) Many merge tools come with an associated diff tool. Some popular merge tools include AraxisMerge (<http://www.araxis.com>) and WinMerge (<http://winmerge.org>). Perforce also comes with an excellent three-way merge tool (<http://www.perforce.com/perforce/products/merge.html>).

- *Hex editors.* A hex editor is a program used for inspecting and modifying the contents of binary files. The data are usually displayed as integers in hexadecimal format, hence the name. Most good hex editors can display data as integers from one byte to 16 bytes each, in 32- and 64-bit floating-point format and as ASCII text. Hex editors are particularly useful when tracking down problems with binary file formats or when reverse-engineering an unknown binary format—both of which are relatively common endeavors in game engine development circles. There are quite literally a million different hex editors out there; I've had good luck with HexEdit by Expert Commercial Software (<http://www.expertcomsoft.com/index.html>), but your mileage may vary.

As a game engine programmer you will undoubtedly come across other tools that make your life easier, but I hope this chapter has covered the main tools you'll use on a day-to-day basis.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

3

Fundamentals of Software Engineering for Games

In this chapter, we'll discuss the foundational knowledge needed by any professional game programmer. We'll explore numeric bases and representations, the components and architecture of a typical computer and its CPU, machine and assembly language, and the C++ programming language. We'll review some key concepts in object-oriented programming (OOP), and then delve into some advanced topics that should prove invaluable in any software engineering endeavor (and especially when creating games). As with Chapter 2, some of this material may already be familiar to some readers. However, I highly recommend that all readers at least skim this chapter, so that we all embark on our journey with the same set of tools and supplies.

3.1 C++ Review and Best Practices

Because C++ is arguably the most commonly used language in the game industry, we will focus primarily on C++ in this book. However, most of the concepts we'll cover apply equally well to *any* object-oriented programming language. Certainly a great many other languages are used in the game industry—imperative languages like C; object-oriented languages like C# and